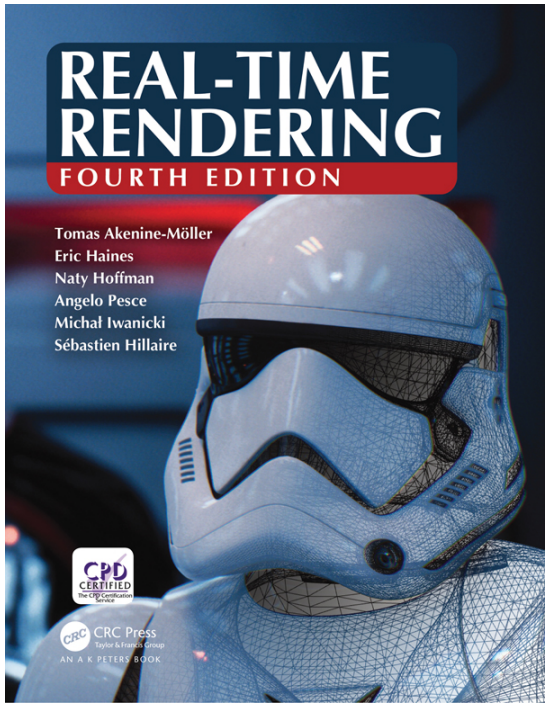
DHBW
Duale Hochschule
Baden-Württemberg

Rasterization Pipeline

Introduction to Computer Graphics - Lecture 03

Pirmin Pfeifer

Literature Reference



Real-Time Rendering,
Fourth Edition
T. Akenine-Möller Et Al.

View Online

Related Chapters:

- 2 The Graphics Pipeline
- 3 The Graphics Processing Unit

A deep dive on the topic can be found in the book ”A trip through the Graphics Pipeline”.

Concept: Pipeline

DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

Concept: Pipeline

DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

- Algorithm that consist out of multiple sub task (stages)

Concept: Pipeline

DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

- Algorithm that consist out of multiple sub task (stages)
- Data is handed from step to step

Concept: Pipeline

DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

- Algorithm that consist out of multiple sub task (stages)
- Data is handed from step to step
- Stages depend linearly on each other (per item/data)

Concept: Pipeline

DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

- Algorithm that consist out of multiple sub task (stages)
- Data is handed from step to step
- Stages depend linearly on each other (per item/data)
- Stages can typically be executed in parallel

Concept: Pipeline

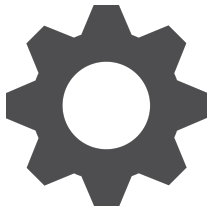
DHBW
Duale Hochschule
Baden-Württemberg

Pipeline

- Algorithm that consist out of multiple sub task (stages)
- Data is handed from step to step
- Stages depend linearly on each other (per item/data)
- Stages can typically be executed in parallel

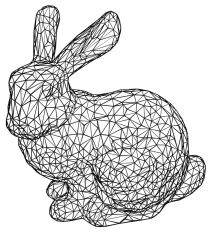
When a pipeline is executed in parallel, the slowest stage determines the throughput of the pipeline and is called the bottleneck.

The Graphics Pipeline[1]

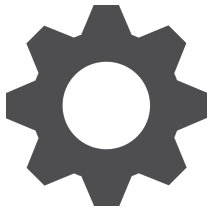


Graphics Pipeline

The Graphics Pipeline[1]

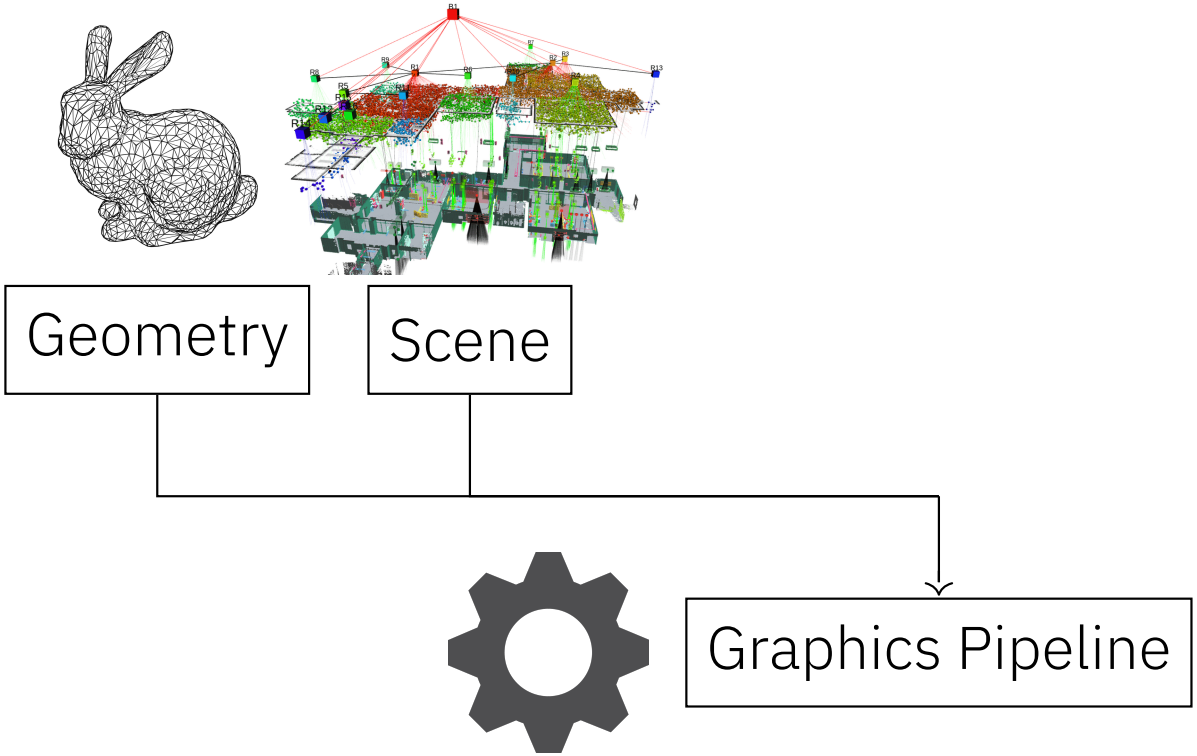


Geometry

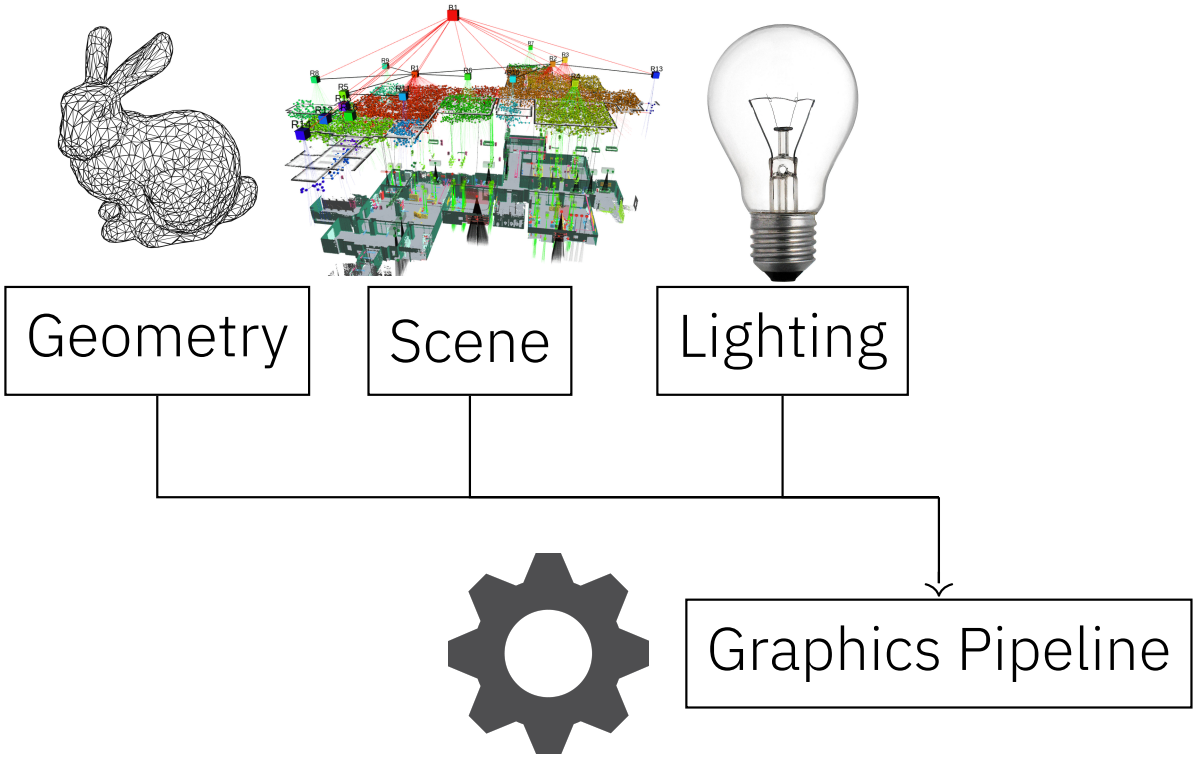


Graphics Pipeline

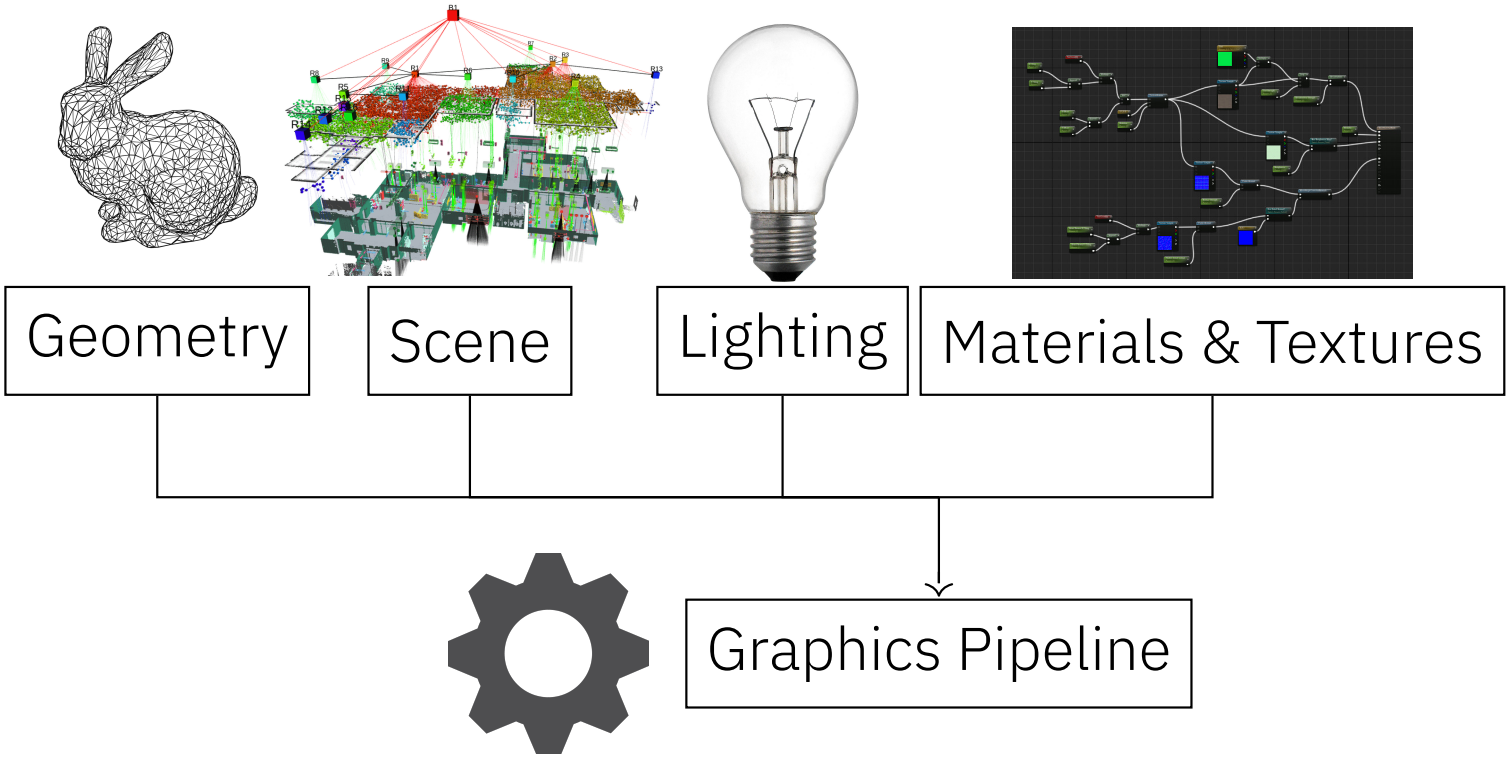
The Graphics Pipeline[1]



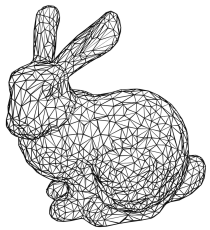
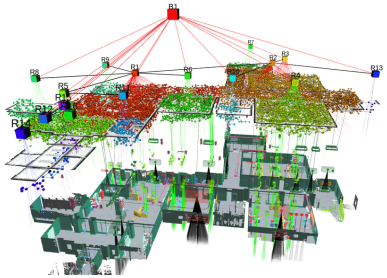
The Graphics Pipeline[1]

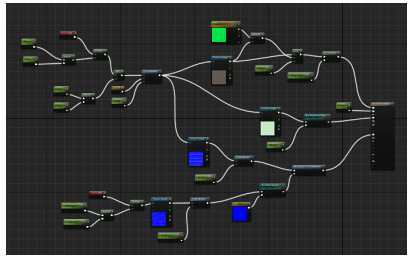
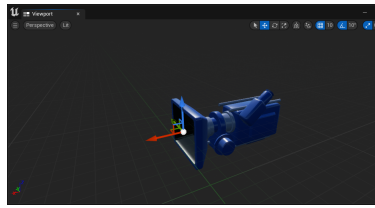


The Graphics Pipeline[1]

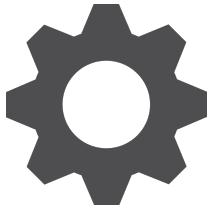


The Graphics Pipeline[1]

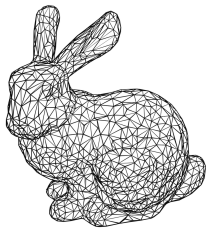
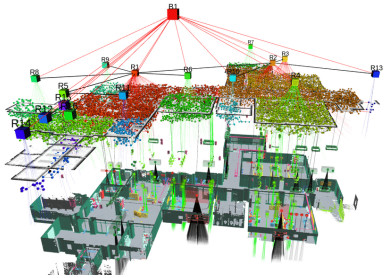



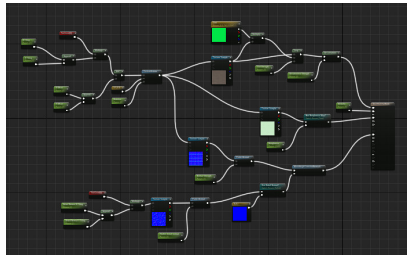
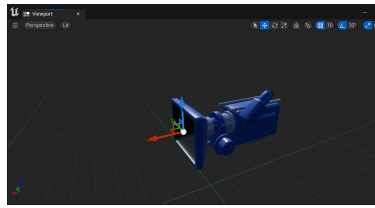
Geometry | Scene | Lighting | Materials & Textures | Camera



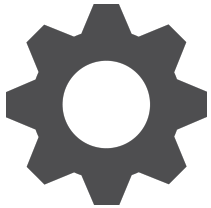
Graphics Pipeline

The Graphics Pipeline[1]

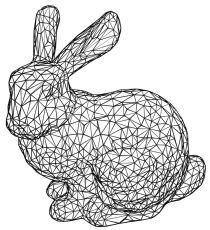
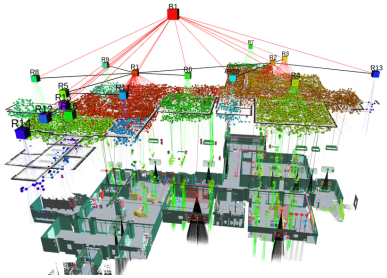
Geometry | Scene | Lighting | Materials & Textures | Camera

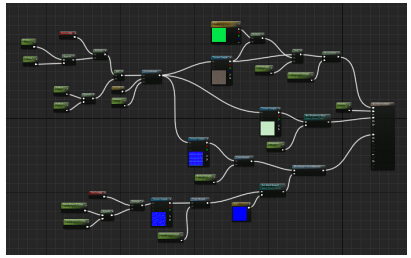
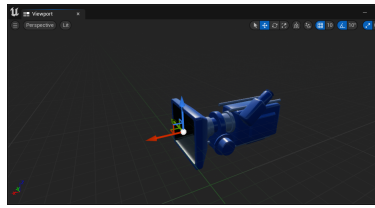


Graphics Pipeline

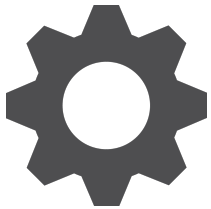
[1] Graph not exhaustive.

The Graphics Pipeline[1]

Geometry | Scene | Lighting | Materials & Textures | Camera



Graphics Pipeline

Rendered Image

[1] Graph not exhaustive.

Geometry

The pipeline is specialized to process vertices and triangle

Geometry

The pipeline is specialized to process vertices and triangle

Vertex
A point in 3D space

Geometry

The pipeline is specialized to process vertices and triangle

Vertex

A point in 3D space

Triangle

Three ordered vertices define a triangle together

Geometry

DHBW
Duale Hochschule
Baden-Württemberg

The pipeline is specialized to process vertices and triangle

Vertex

A point in 3D space

Triangle

Three ordered vertices define a triangle together

Mesh

A collection of (mostly connected) triangles

Concept: Shader

DHBW
Duale Hochschule
Baden-Württemberg

Shader

A program that runs on the GPU.

Concept: Shader

DHBW
Duale Hochschule
Baden-Württemberg

Shader

A program that runs on the GPU. It is commenly used in the Graphics pipeline to:

- Transform and calculate vertex-positions

Concept: Shader

DHBW
Duale Hochschule
Baden-Württemberg

Shader

A program that runs on the GPU. It is commenly used in the Graphics pipeline to:

- Transform and calculate vertex-positions
- Determine pixel-colors

Concept: Shader

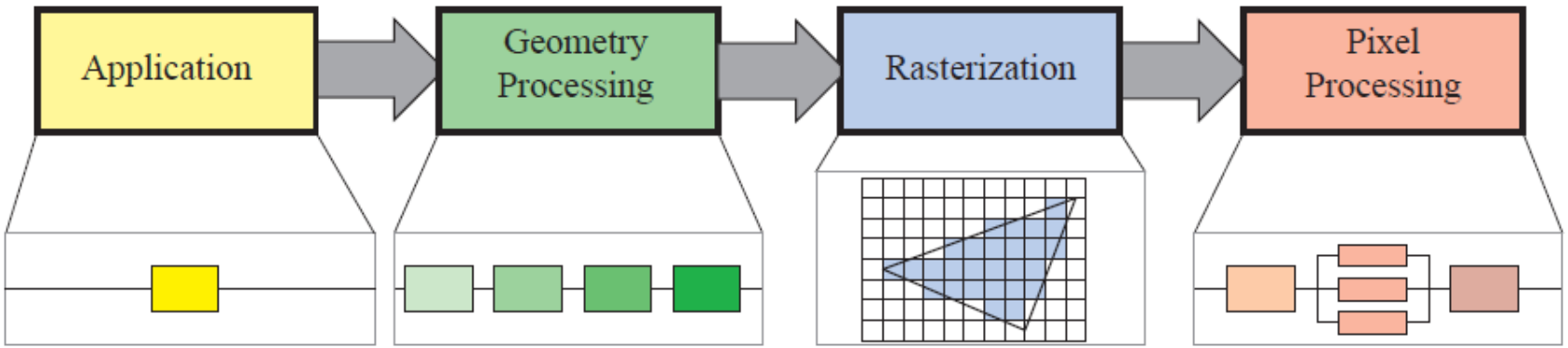
DHBW
Duale Hochschule
Baden-Württemberg

Shader

A program that runs on the GPU. It is commenly used in the Graphics pipeline to:

- Transform and calculate vertex-positions
- Determine pixel-colors
- Do arbitrary calculations

The Graphics Pipeline - Stages

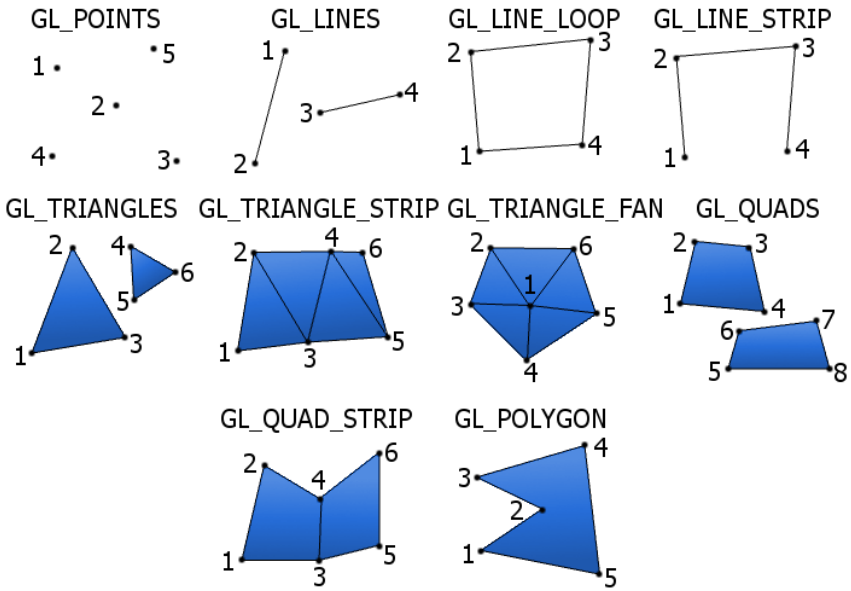


Application Stage

- Dispatch *rendering primitives* into the pipeline (Draw Calls)

Application Stage

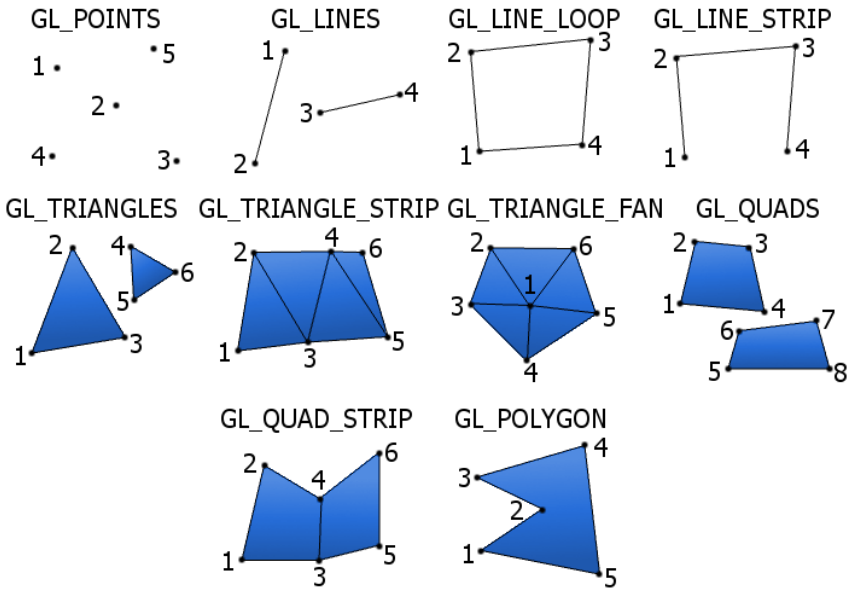
- Dispatch *rendering primitives* into the pipeline (Draw Calls)



OpenGl Rendering Primitives

Application Stage

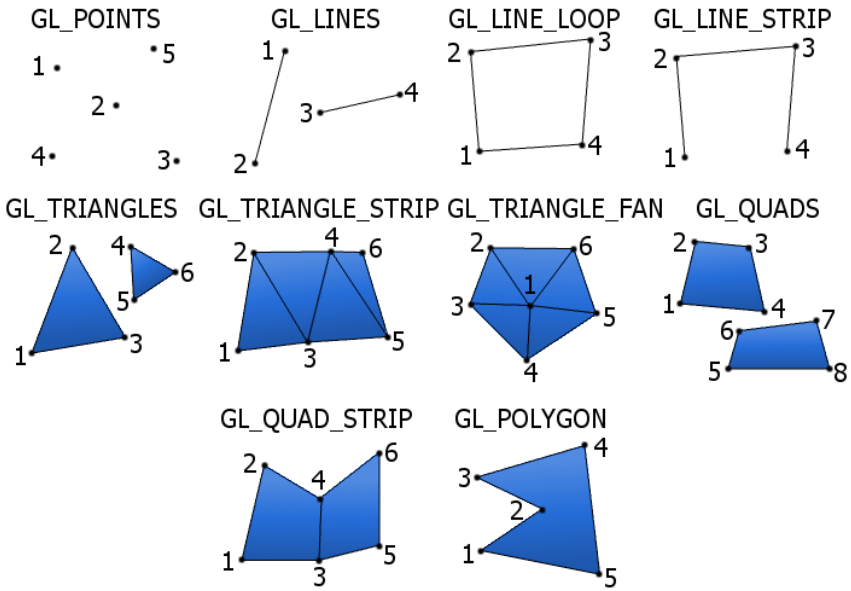
- Dispatch *rendering primitives* into the pipeline (Draw Calls)
- Configure later pipeline stages



OpenGl Rendering Primitives

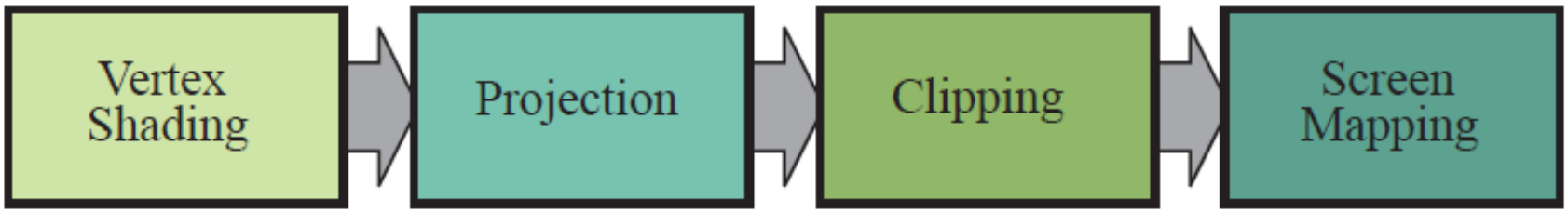
Application Stage

- Dispatch *rendering primitives* into the pipeline (Draw Calls)
- Configure later pipeline stages
- Done mostly on CPU



OpenGl Rendering Primitives

Geometry Processing Stage



The Geometry Processing Stage operates on a per vertex basis.

Geometry Stage: Vertex Shading

Duale Hochschule Baden-Württemberg

Main Tasks:

Geometry Stage: Vertex Shading

Duale Hochschule Baden-Württemberg

Main Tasks:

- Transform vertex position[2]

[2] This can include blend and morph animation

Geometry Stage: Vertex Shading

Duale Hochschule Baden-Württemberg

Main Tasks:

- Transform vertex position[2]

[2] This can include blend and morph animation

Geometry Stage: Vertex Shading

Main Tasks:

- Transform vertex position[2]

- Evaluate all other vertex-attributes[3] such as:

[2] This can include blend and morph animation

[3] These can be chosen freely by the programmer

Geometry Stage: Vertex Shading

Duale Hochschule Baden-Württemberg

Main Tasks:

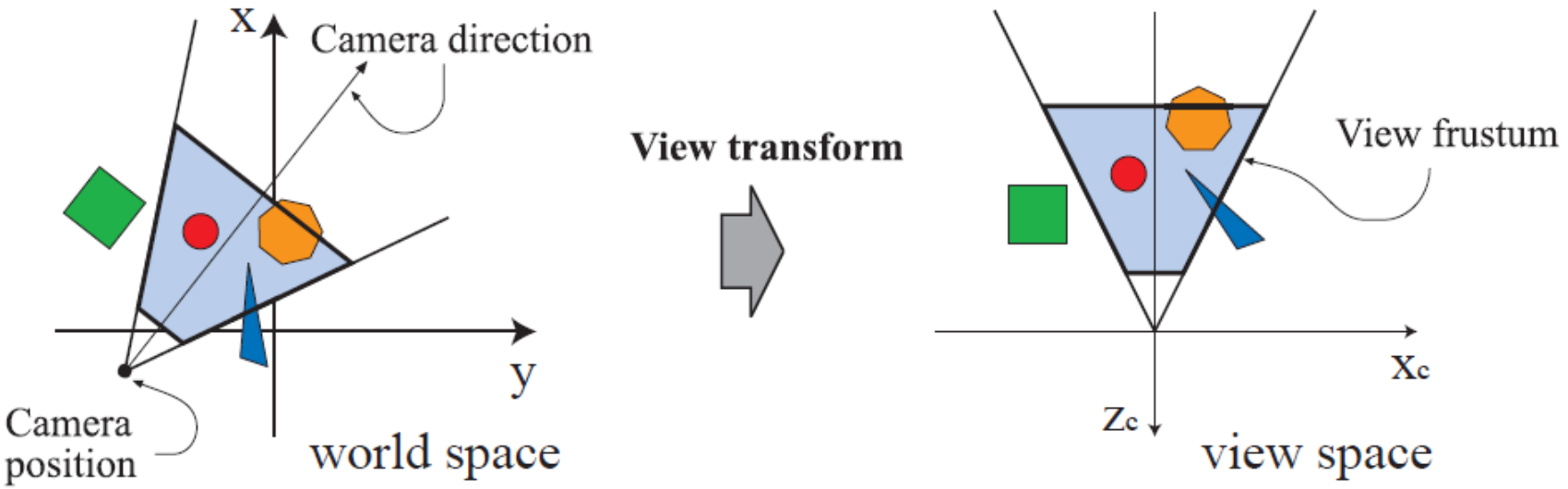
- Transform vertex position[2]

- Evaluate all other vertex-attributes[3] such as:
  - Normals
  - Texture-Coordinates
  - Vertex-Color

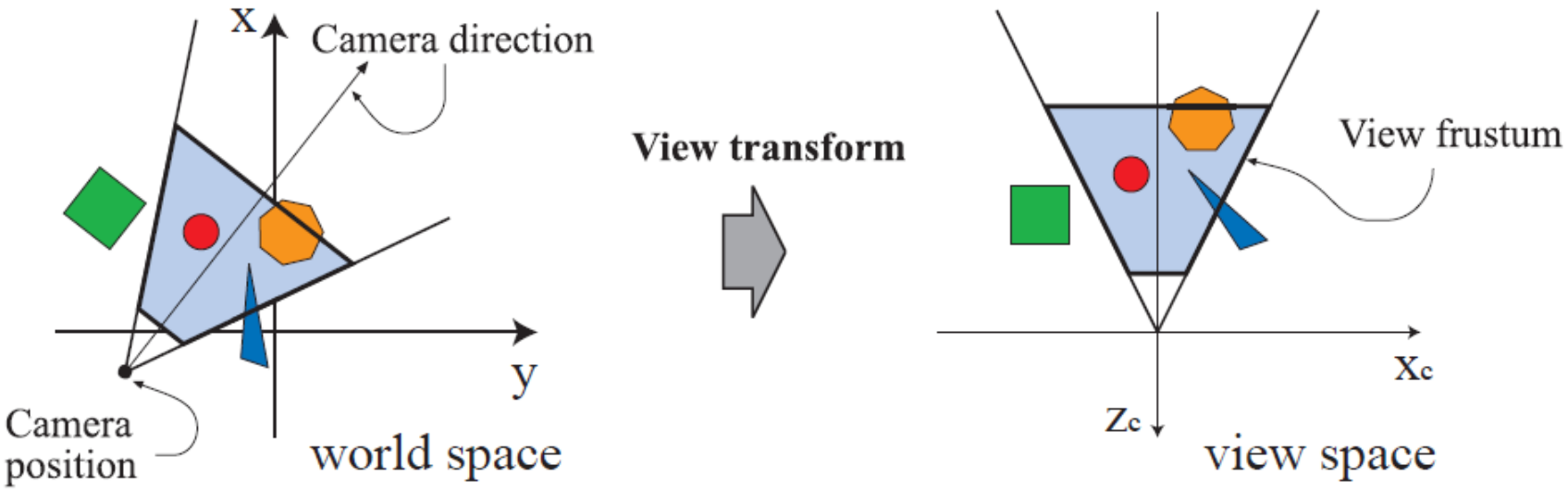
[2] This can include blend and morph animation

[3] These can be chosen freely by the programmer

Geometry Stage: Vertex Shading



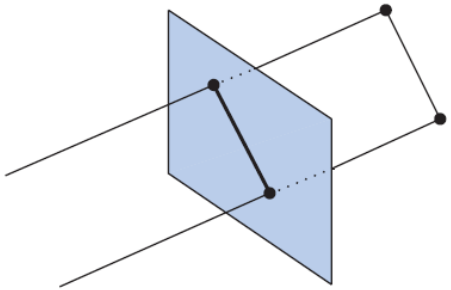
Geometry Stage: Vertex Shading

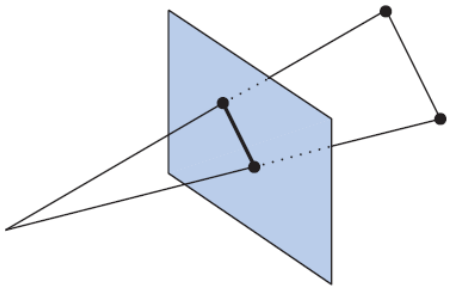


default.vert

```
1   #version 330 core
2   layout (location = 0) in vec3 aPos;
3
4   void main()
5   {
6       gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
7   }
```

Geometry Stage: Projection

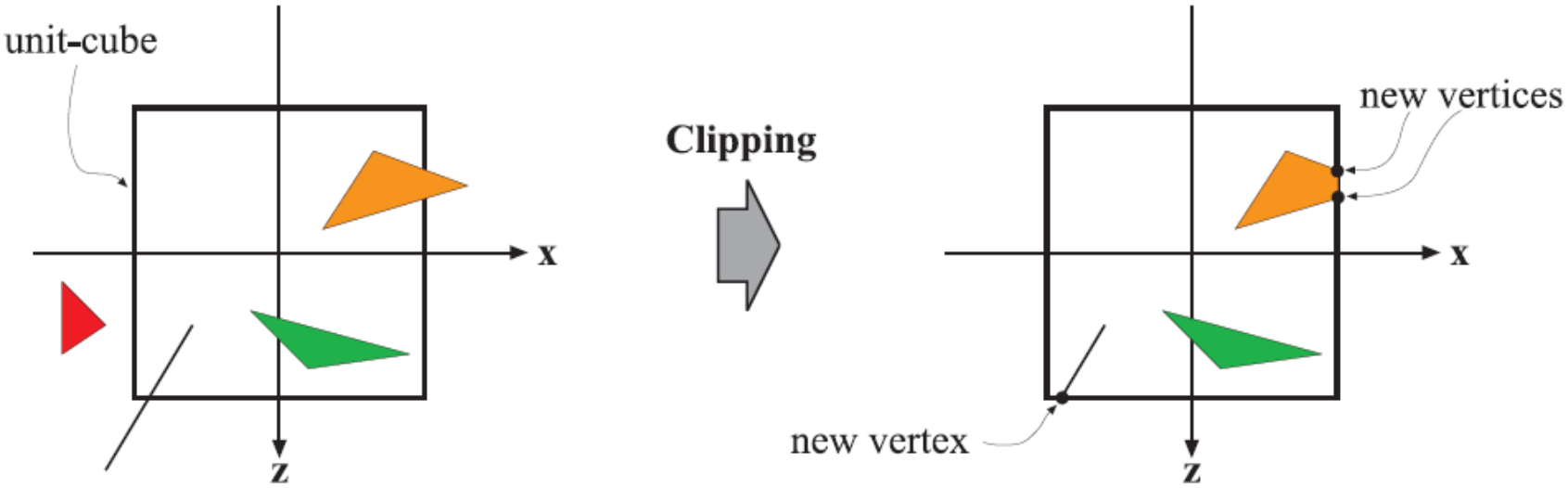




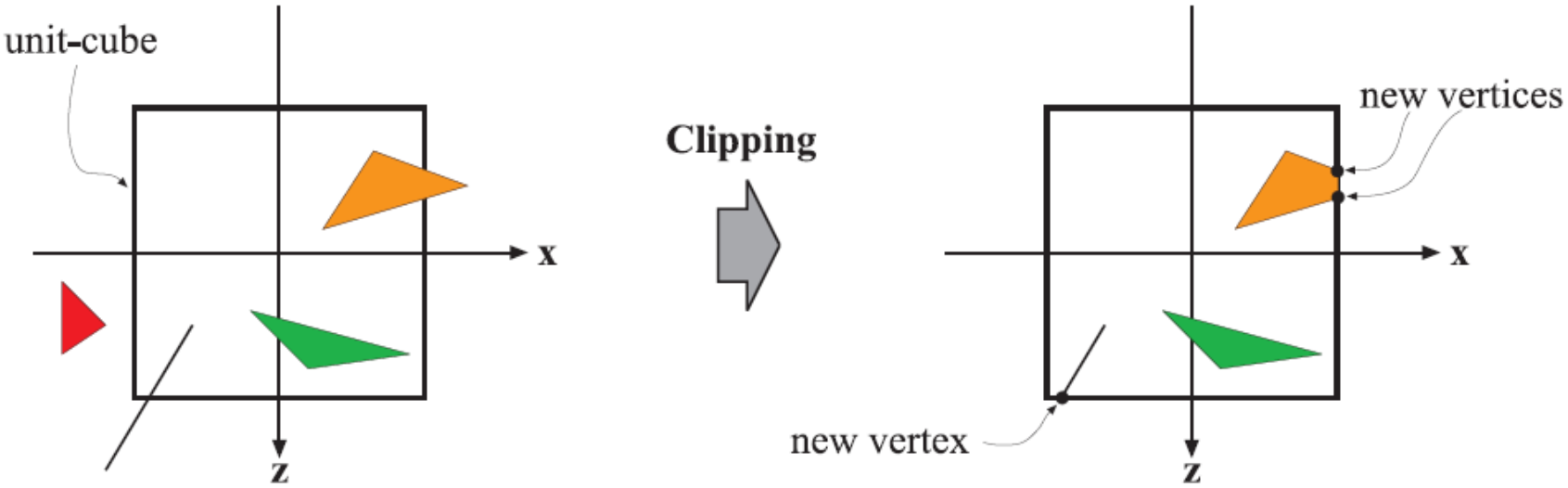
Orthographic Projection

Perspective Projection

Geometry Stage: Clipping

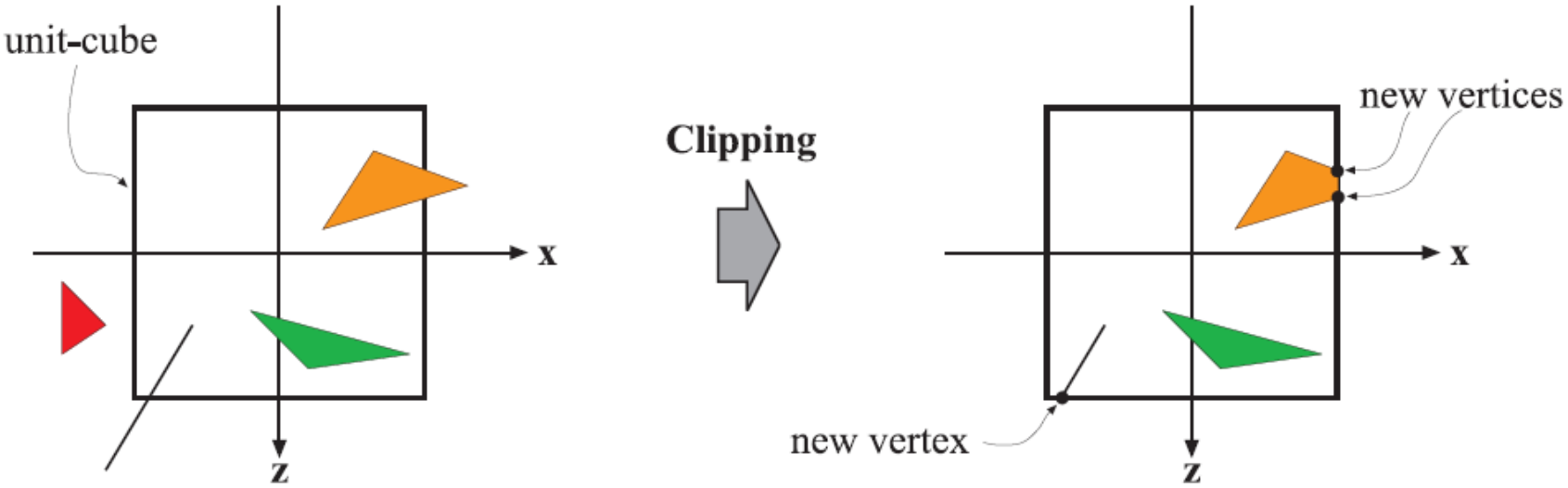


Geometry Stage: Clipping



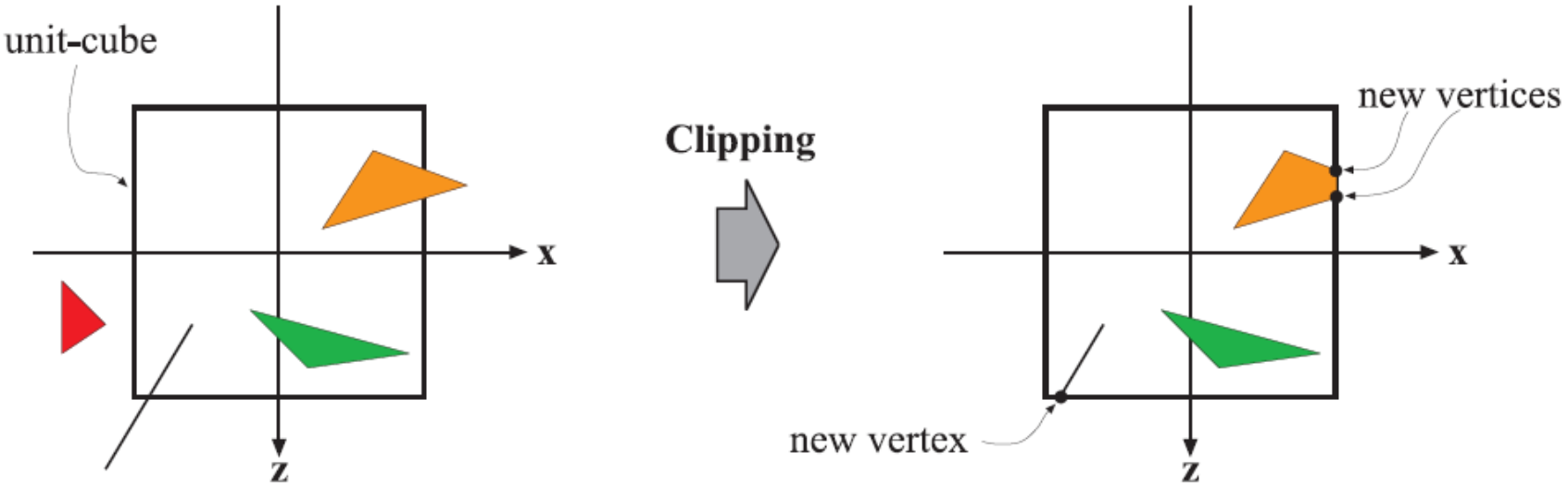
- Cull vertices which are outside of the unit cube

Geometry Stage: Clipping



- Cull vertices which are outside of the unit cube
- At edges of cube new vertices are added to replace removed

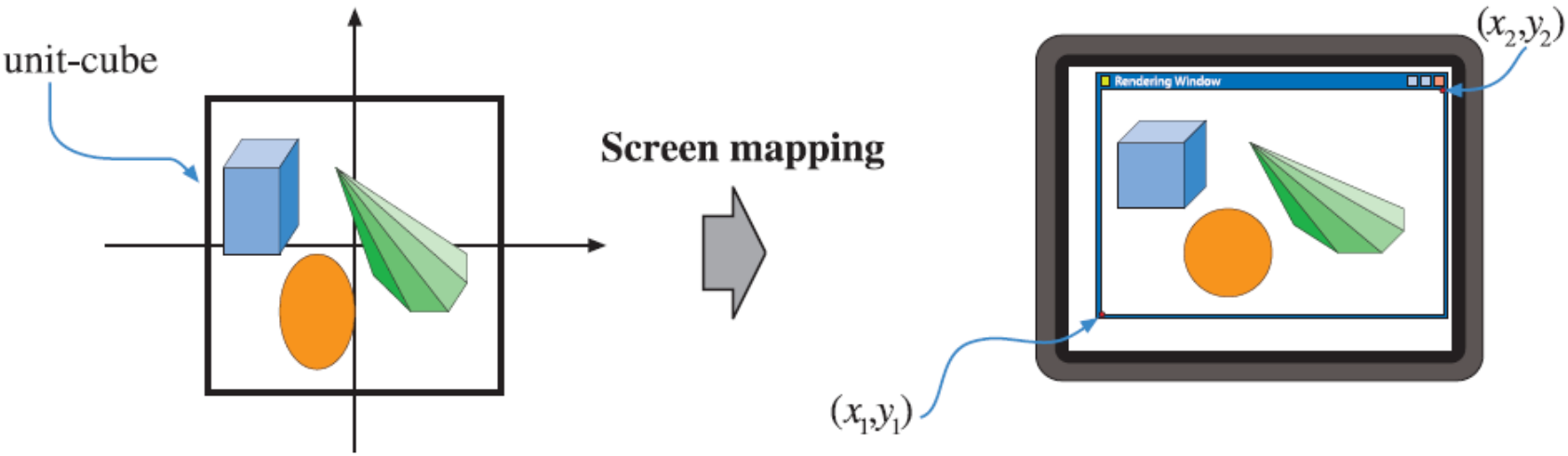
Geometry Stage: Clipping



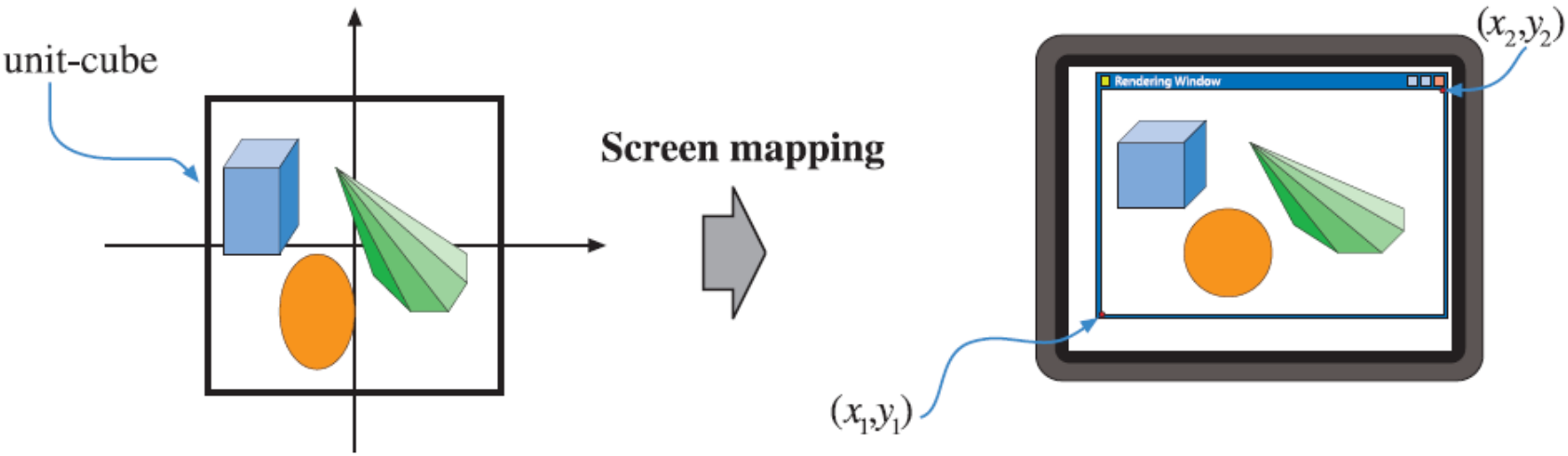
- Cull vertices which are outside of the unit cube
- At edges of cube new vertices are added to replace removed

⇒ Only continue with vertices which contribute to the image

Geometry Stage: Screen Mapping

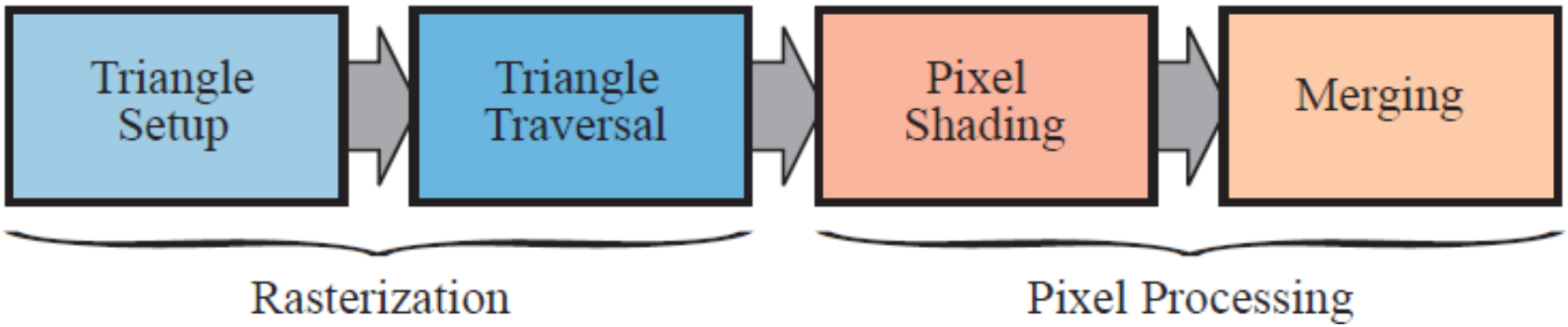


Geometry Stage: Screen Mapping



Convert from *View Space* to *Screen Space* as last step of the *Geometry Processing Stage*

Rasterization and Pixel Processing



Rasterization: Primitive Assembly[5]

- Assembles all data from the Geometry Stage into batches of triangles for the following stages

[5] Also called *Triangle Setup*

Rasterization: Primitive Assembly[5]

DHBW
Duale Hochschule
Baden-Württemberg

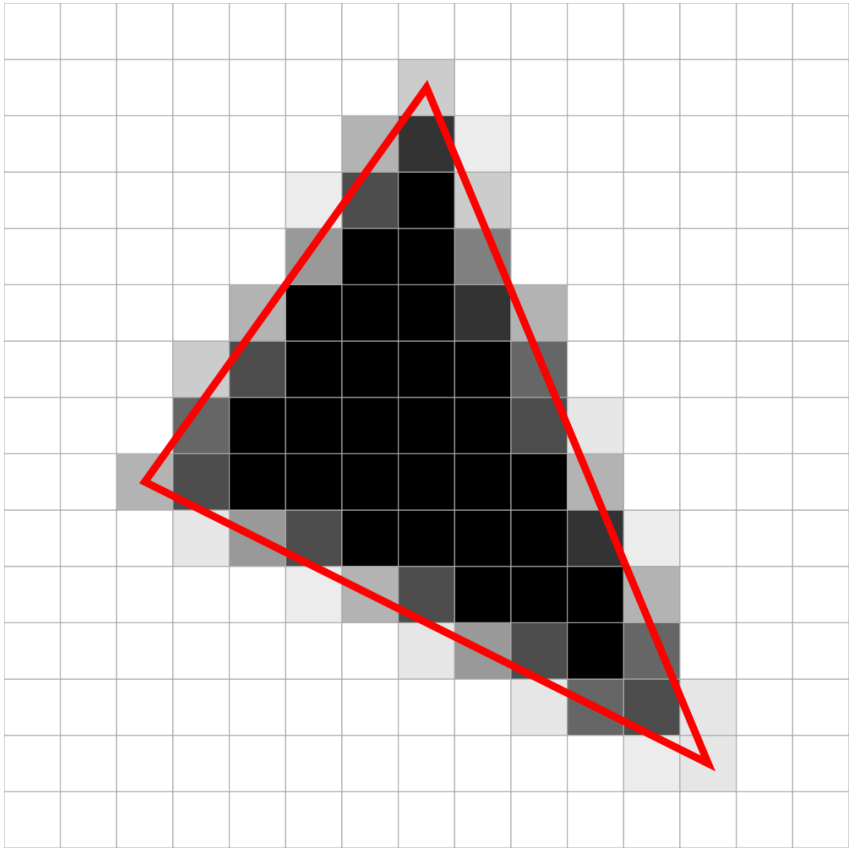
- Assembles all data from the Geometry Stage into batches of triangles for the following stages
- Uses fixed function[4] hardware

[4] non programmable hardware functions

[5] Also called *Triangle Setup*

Rasterization: Triangle Traversal

- Traversing through each triangle generating a *fragment* for each overlapping Pixel



Pixel Shading

- A Fragment Shader is applied to each fragment

Pixel Shading

DHBW
Duale Hochschule
Baden-Württemberg

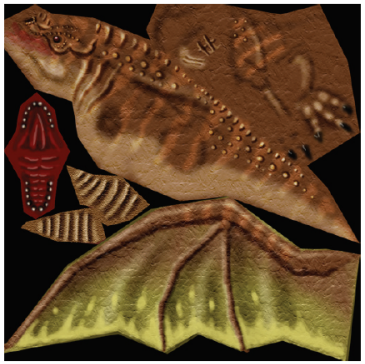
- A Fragment Shader is applied to each fragment
- Uses interpolated data from the vertices of the triangle

Pixel Shading

- A Fragment Shader is applied to each fragment
- Uses interpolated data from the vertices of the triangle
- Passes a Color and Depth to the next stage

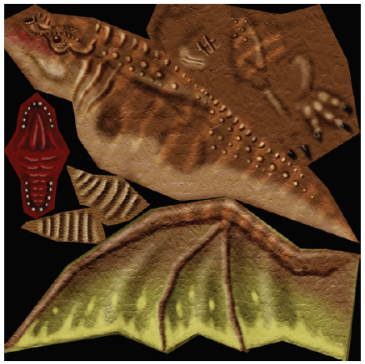
Pixel Shading

- A Fragment Shader is applied to each fragment
- Uses interpolated data from the vertices of the triangle
- Passes a Color and Depth to the next stage



Pixel Shading

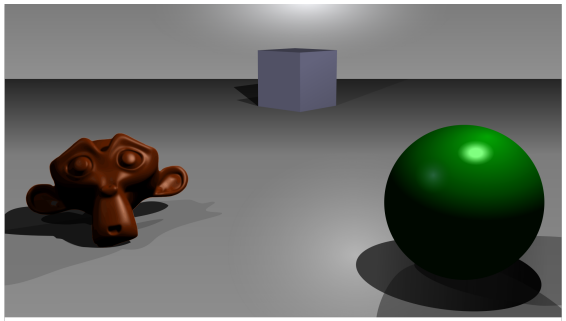
- A Fragment Shader is applied to each fragment
- Uses interpolated data from the vertices of the triangle
- Passes a Color and Depth to the next stage



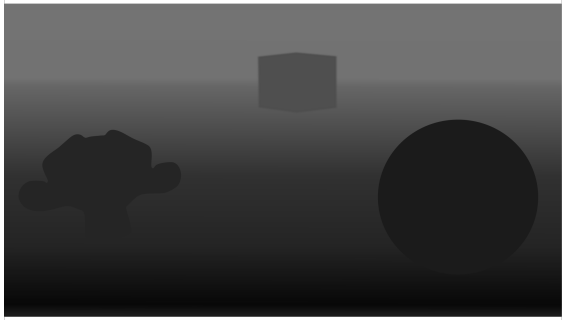
default.frag

```
1  #version 330 core
2  out vec4 FragColor;
3
4  void main()
5  {
6      FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
7  }
```

Merging

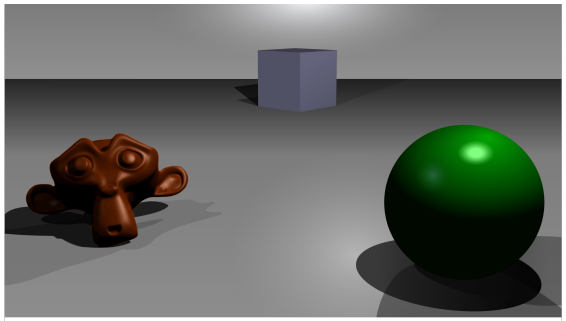


A simple three-dimensional scene

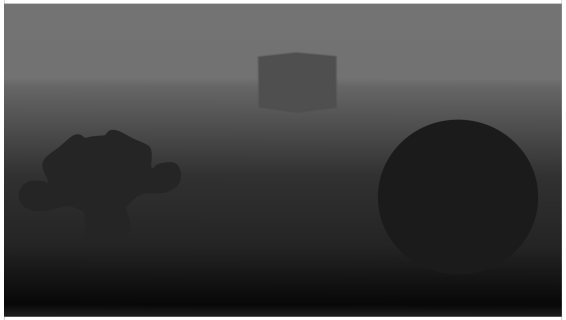


Z-buffer representation

Merging



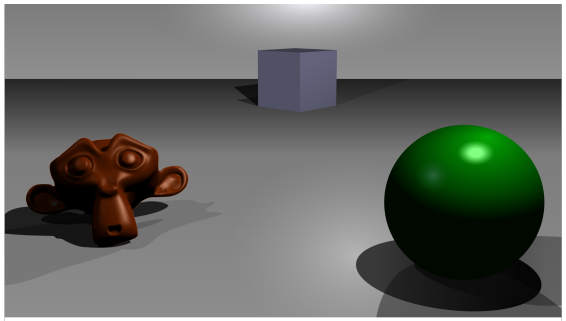
A simple three-dimensional scene



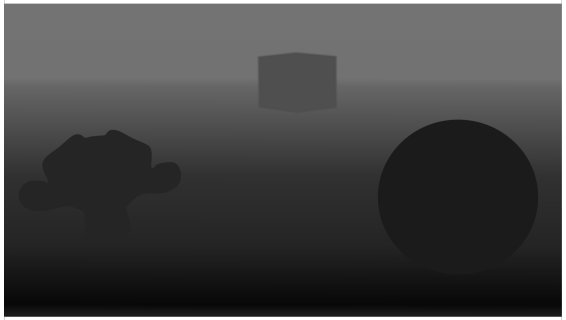
Z-buffer representation

- Merge Pixel Shading result into the frame buffer based on depth

Merging



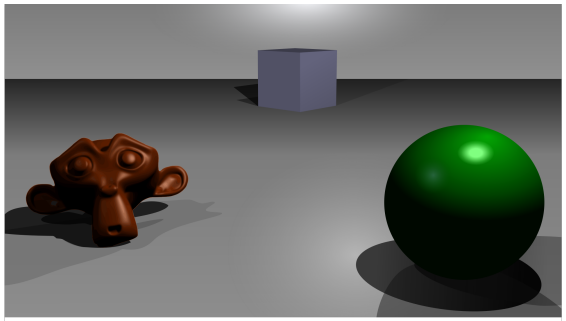
A simple three-dimensional scene



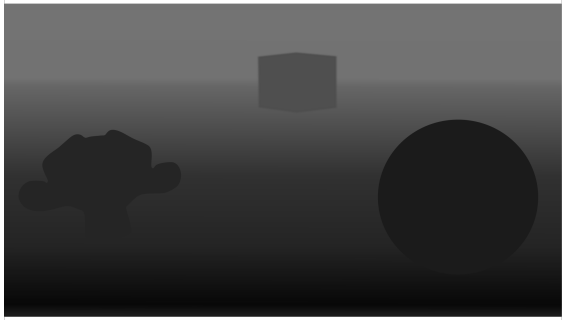
Z-buffer representation

- Merge Pixel Shading result into the frame buffer based on depth
- Depth check can be made early to reduce performance cost fo discarded fragments

Merging



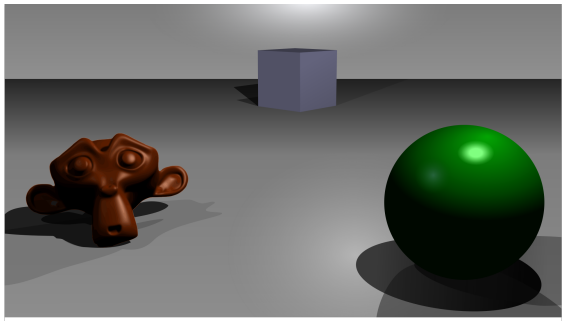
A simple three-dimensional scene



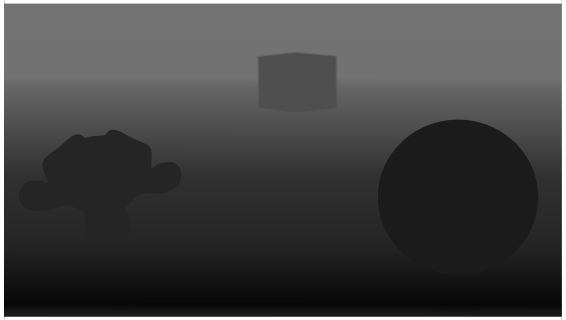
Z-buffer representation

- Merge Pixel Shading result into the frame buffer based on depth
- Depth check can be made early to reduce performance cost fo discarded fragments
- Each additional write to a pixel in the frame buffer is called overdraw

Merging



A simple three-dimensional scene



Z-buffer representation

- Merge Pixel Shading result into the frame buffer based on depth
- Depth check can be made early to reduce performance cost fo discarded fragments
- Each additional write to a pixel in the frame buffer is called overdraw
- Multiple Buffers maybe used to pass data to postprocessing

Parallel Possibilities

- The whole pipeline can be parallelized

Parallel Possibilities

- The whole pipeline can be parallelized
- The execution order of the individual vertices, triangles and pixels can be chose arbitrarily

Parallel Possibilities

- The whole pipeline can be parallelized
- The execution order of the individual vertices, triangles and pixels can be chose arbitrarily
- The whole pipeline can implement and executed on a CPU but there is alot of work to be done

Parallel Possibilities

- The whole pipeline can be parallelized
- The execution order of the individual vertices, triangles and pixels can be chose arbitrarily
- The whole pipeline can implement and executed on a CPU but there is alot of work to be done

$\Rightarrow$ Maybe parallel hardware is needed

How much Work needs to be done?

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

[6] In million pixels per second

How much Work needs to be done?

Duale Hochschule Baden-Württemberg

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s[6]	ms/frame

[6] In million pixels per second

How much Work needs to be done?

Duale Hochschule Baden-Württemberg

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s[6]	ms/frame
1080P@30	62.2 MP	33.3 ms
1080P@60	124.5 MP	16.6 ms
1080P@144	298.5 MP	6.9 ms
1080P@240	497.6 MP	4.2 ms

[6] In million pixels per second

How much Work needs to be done?

Duale Hochschule Baden-Württemberg

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s[6]	ms/frame
1080P@30	62.2 MP	33.3 ms
1080P@60	124.5 MP	16.6 ms
1080P@144	298.5 MP	6.9 ms
1080P@240	497.6 MP	4.2 ms
1440P@30	110.5 MP	33.3 ms
1440P@60	221.2 MP	16.6 ms
1440P@144	530.8 MP	6.9 ms

[6] In million pixels per second

How much Work needs to be done?

Duale Hochschule Baden-Württemberg

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s[6]	ms/frame
1080P@30	62.2 MP	33.3 ms
1080P@60	124.5 MP	16.6 ms
1080P@144	298.5 MP	6.9 ms
1080P@240	497.6 MP	4.2 ms
1440P@30	110.5 MP	33.3 ms
1440P@60	221.2 MP	16.6 ms
1440P@144	530.8 MP	6.9 ms
2160P@30	248.8 MP	33.3 ms
2160P@60	497.6 MP	16.6 ms

[6] In million pixels per second

How much Work needs to be done?

Duale Hochschule Baden-Württemberg

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s[6]	ms/frame
1080P@30	62.2 MP	33.3 ms
1080P@60	124.5 MP	16.6 ms
1080P@144	298.5 MP	6.9 ms
1080P@240	497.6 MP	4.2 ms
1440P@30	110.5 MP	33.3 ms
1440P@60	221.2 MP	16.6 ms
1440P@144	530.8 MP	6.9 ms
2160P@30	248.8 MP	33.3 ms
2160P@60	497.6 MP	16.6 ms

⇒ Specialized parallel hardware is needed

[6] In million pixels per second

The GPU

CPU **vs** **GPU**

The GPU

CPU	vs	GPU
4-32	Cores	500-15000+

GPU cores refers to CUDA Cores (which are not the completly compareable to CPU cores).

The GPU

CPU	vs	GPU
4-32	Cores	500-15000+
4-8	SIMD	8-64

GPU cores refers to CUDA Cores (which are not the completly compareable to CPU cores).

The GPU

CPU	vs	GPU
4-32	Cores	500-15000+
4-8	SIMD	8-64
5-6 GHz	Frequency	2 GHz

GPU cores refers to CUDA Cores (which are not the completly compareable to CPU cores).

The GPU

CPU	vs	GPU
4-32	Cores	500-15000+
4-8	SIMD	8-64
5-6 GHz	Frequency	2 GHz
branch prediction cache prefetching	latency- strategy	quick context switch

GPU cores refers to CUDA Cores (which are not the completly compareable to CPU cores).

The GPU

CPU	vs	GPU
4-32	Cores	500-15000+
4-8	SIMD	8-64
5-6 GHz	Frequency	2 GHz
branch prediction cache prefetching	latency- strategy	quick context switch

Modern GPU can handle shader Invocations with over a million threads.

GPU cores refers to CUDA Cores (which are not the completly compareable to CPU cores).

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

- The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

- The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD
- With a 32 Thread SIMD core we need 62.5 wavefronts for the shader

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

- The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD
- With a 32 Thread SIMD core we need 62.5 wavefronts for the shader
- The shader is executed 63 times with a width of 32

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

- The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD
- With a 32 Thread SIMD core we need 62.5 wavefronts for the shader
- The shader is executed 63 times with a width of 32
- When a fetch is encountered a new wavefront is started on the core unit the fetch is completed

The GPU - an example

Duale Hochschule Baden-Württemberg

Lets say we need to execute 2000 fragments for one triangle:

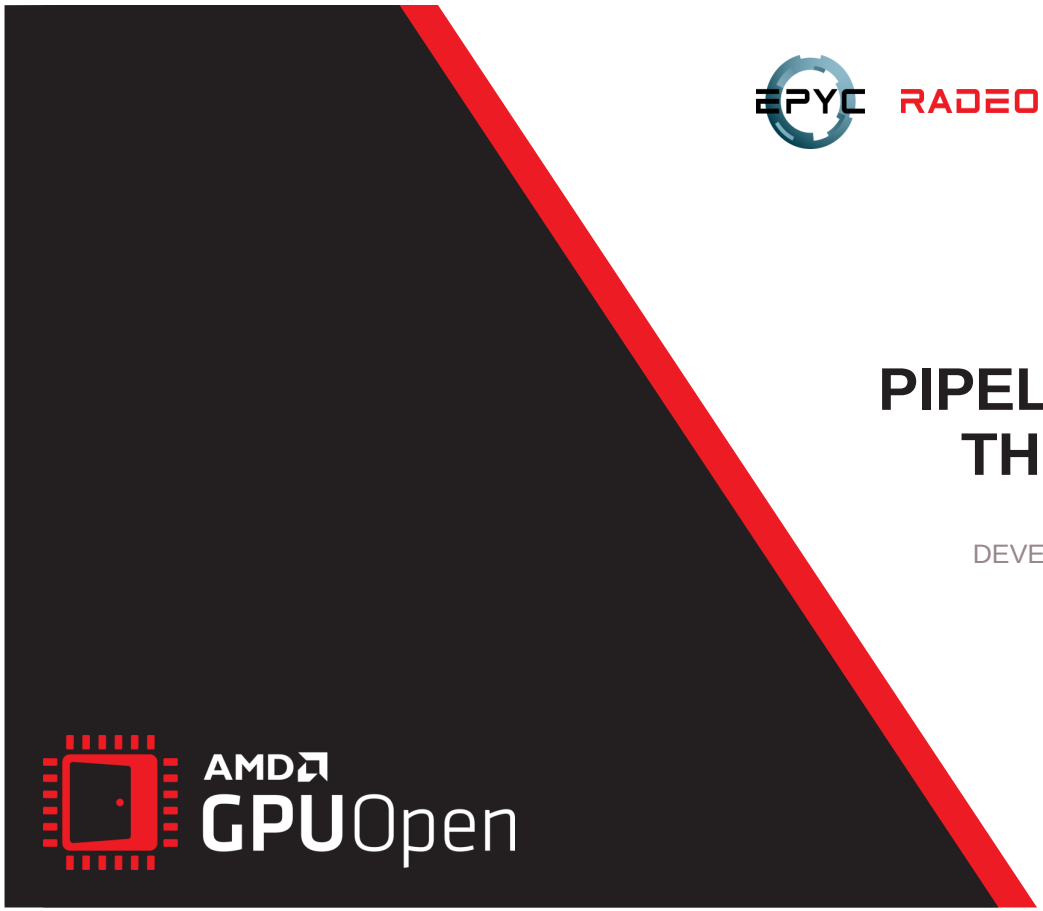
- The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD
- With a 32 Thread SIMD core we need 62.5 wavefronts for the shader
- The shader is executed 63 times with a width of 32
- When a fetch is encountered a new wavefront is started on the core unit the fetch is completed

The occupancy of a GPU is measured by the number of wavefronts *in flight* on the GPU.

The GPU - Hardware

DHBW
Duale Hochschule
Baden-Württemberg

For a deeper dive into morder GPU Hardware you can take a look at this Presentation from 2020 by AMD:



Any Questions?